DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 98

Question description

You are given an array *A* of length *N* which is initialised with *0*. You will be given *Q* queries of two types:

1 k: set value *1* at index *k* in array *A*

2 y: print the smallest index *x* which is greater than or equal to *y* and having value *1*. If there is no such index print *1*.

Note: Indexing is *1* based

Constraints

$1 \leq n \leq 10^9$
$1 \leq q \leq 5 * 10^5$
$1 \leq y, k \leq n$

Input Format

First line contains two integers *N* and *Q* separated by a space.

The next *Q* lines contain the type of query (i.e. either a *1* or a *2*), then a space, then for type *1* queries integer *k* and for type *2* queries integer *y*

Output Format

For each query type *2*, print in new line, the smallest index *x* which is greater than or equal to *y* and having value *1*. If there is no such index print *1*.

Explanation for Test case 1

For first query: 2 3, there is no index greater than or equal index 3, having value -1, so the answer is -1.

For second query: 1 2, set value -1 at index 2.

For third query: 2 1, index 2 is greater than index 1 , having value -1, so the answer is 2.

For fourth query: 2 3, there is no index greater than or equal index 3, having value -1, so the answer is -1.

For fifth query: 2 2, index 2 is equal to index 2 , having value -1, so the answer is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 5
2 3
1 2
2 1
2 3
2 2
```

EXPECTED OUTPUT

```
-1
2
-1
2
```

Test Case 2

INPUT (STDIN)

```
5 5
2 1
1 2
2 2
1 2
2 3
```

EXPECTED OUTPUT

```
-1
2
-1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<q;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

225

Test Case 3

NLOC

38

Code

point_update_lower_bound.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXQ 500005

typedef struct Node {
    int key;
    int priority;
    struct Node *left, *right;
} Node;

Node* new_node(int key) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->key = key;
    n->priority = rand();
    n->left = n->right = NULL;
    return n;
}

Node* rotate_right(Node* y) {
    Node* x = y->left;
    y->left = x->right;
    x->right = y;
    return x;
}

Node* rotate_left(Node* x) {
    Node* y = x->right;
    x->right = y->left;
    y->left = x;
    return y;
}

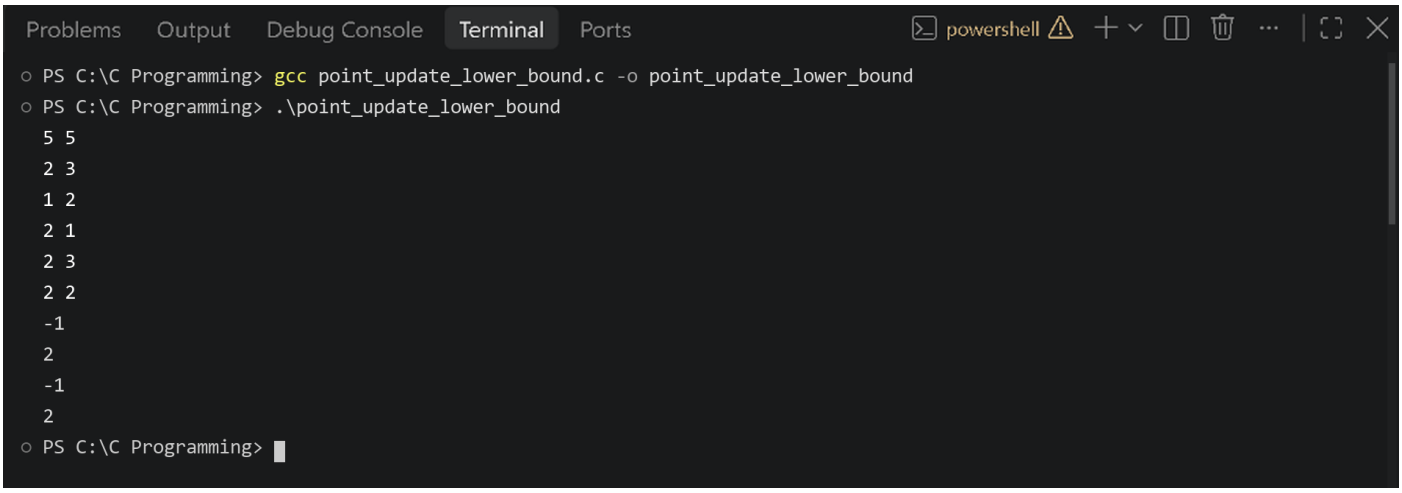
Node* insert(Node* root, int key) {
    if (!root) return new_node(key);
    if (key < root->key) {
        root->left = insert(root->left, key);
        if (root->left->priority > root->priority)
            root = rotate_right(root);
    } else if (key > root->key) {
        root->right = insert(root->right, key);
        if (root->right->priority > root->priority)
            root = rotate_left(root);
    }
    return root;
}

int find_lower_bound(Node* root, int y) {
    int ans = -1;
    while (root) {
        if (root->key >= y) {
            ans = root->key;
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return ans;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

    Node* root = NULL;
    int i;
    for(i=0; i<q; i++) {
        int type, val;
        scanf("%d %d", &type, &val);
        if (type == 1) {
            root = insert(root, val);
        } else {
            printf("%d\n", find_lower_bound(root, val));
        }
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc point_update_lower_bound.c -o point_update_lower_bound
PS C:\C Programming> .\point_update_lower_bound
5 5
2 3
1 2
2 1
2 3
2 2
-1
2
-1
2
PS C:\C Programming>